How-To: Guide to Database Migration from Microsoft SQL Server using MySQL Workbench

MySQL Workbench 5.2.41 introduces a new Migration Wizard module. This module allows you to easily and quickly migrate databases from various RDBMS products to MySQL. In this initial version, migrations from Microsoft SQL Server are supported, but it should also be possible to migrate from most ODBC capable RDBMS as well, using its generic RDBMS support. Additionally, you can use it to perform MySQL to MySQL database copies, which can be used for tasks such as copying a database across servers or migrating data across different versions of MySQL.

So let's get our hands dirty and run through the Migration Wizard in order to migrate a Microsoft SQL Server database to MySQL. In the rest of this post I assume that you have:

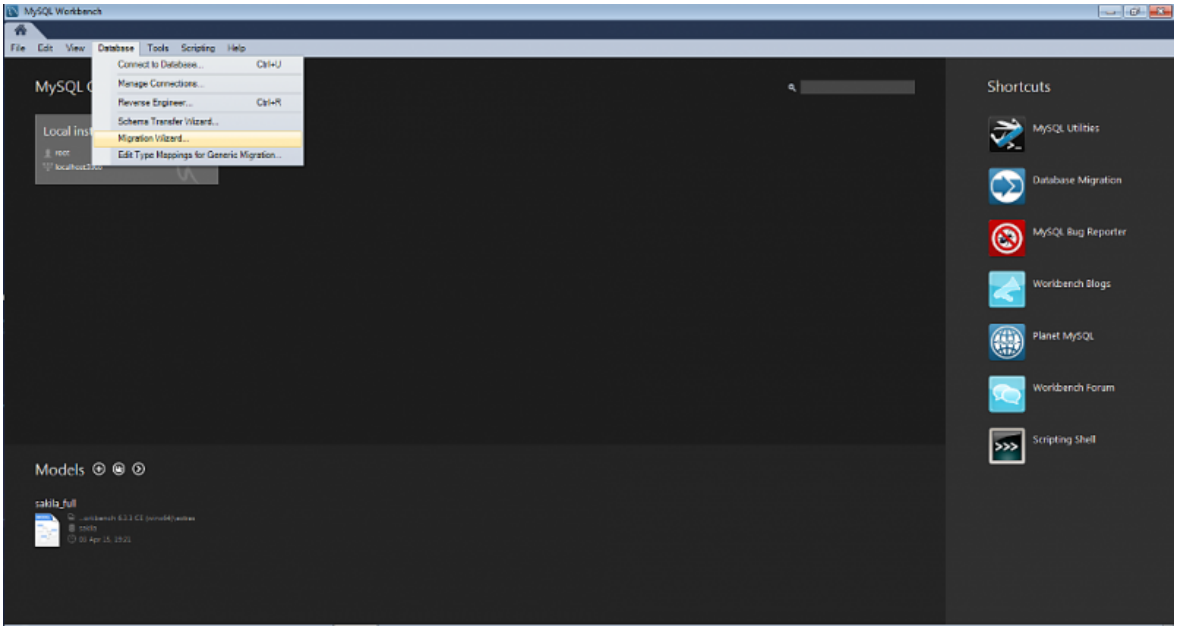
- A running SQL Server instance in which you have proper access to the database you want to migrate. (I'll call this database from now on the *source database*). I have a remote SQL Server 2000 instance available and the sample Northwind database on top of it. I'm using the standard **"sa"** user, which has full privileges. You can use whatever SQL Server version you have at hand. Keep in mind that the Migration Wizard officially supports SQL Server 2000 and newer so older SQL Server versions might not work.
- A running MySQL Server instance with proper user access. The Migration Wizard supports MySQL versions from 5.0 onwards so make sure you have a supported version. For this tutorial I'm using MySQL Server 5.5.15 CE installed in the same PC where MySQL Workbench is running.
- MySQL Workbench 5.2.41 or newer for Windows. The Migration Wizard is also available in the Linux and Mac versions of MySQL Workbench, but running it

from Windows will save us from installing an ODBC driver to connect to our SQL Server instance. Other blog posts will follow on how to proceed in those cases.

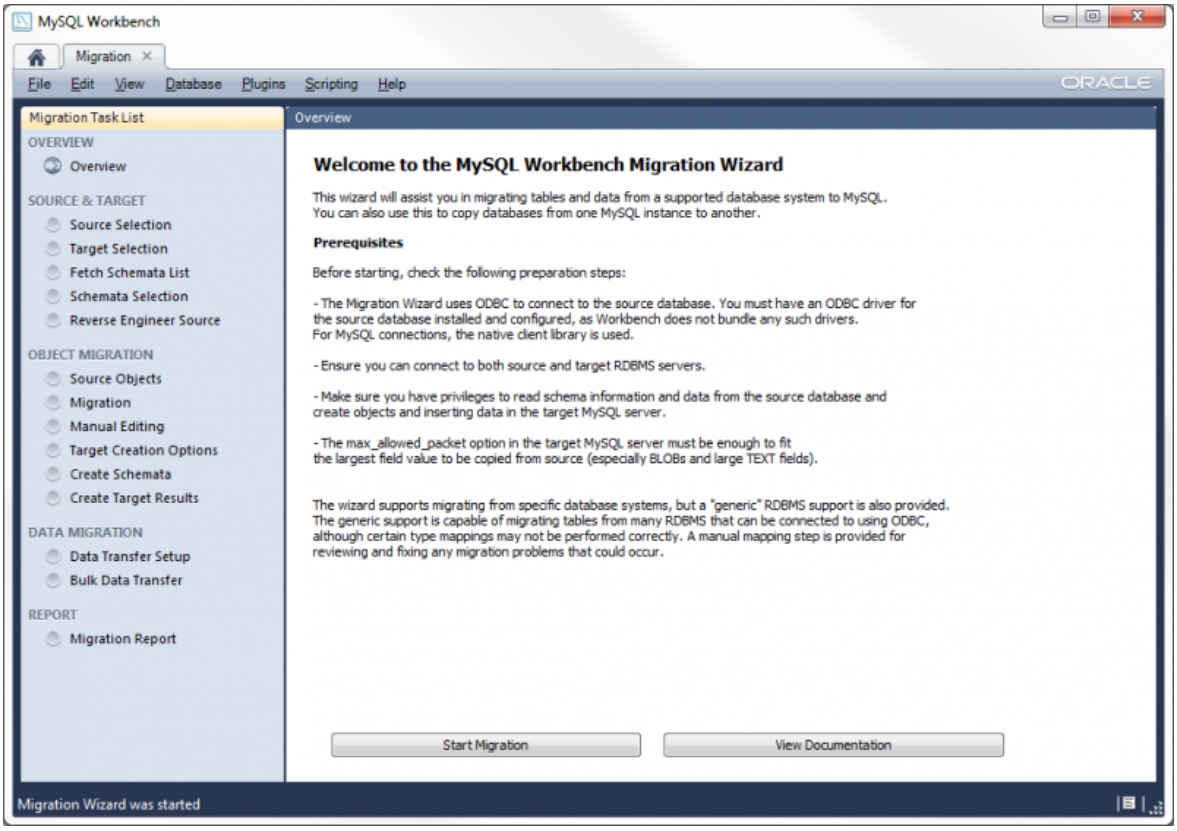
Let's start now...

Open MySQL Workbench and start the Migration Wizard

From the main MySQL Workbench screen you can start the Migration Wizard by clicking on the Database Migration launcher in the Workbench Central panel or through **Database –> Migrate** in the main menu.

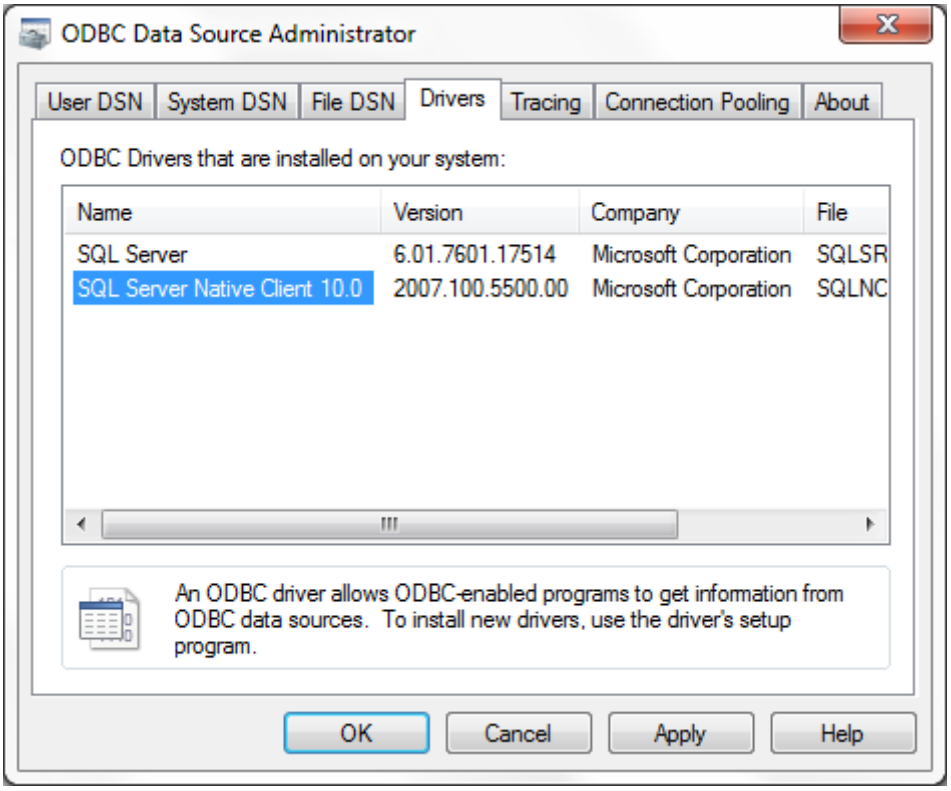


A new tab showing the Overview page of the Migration Wizard should appear.



Read carefully the Prerequisites section. You can read there that you need an ODBC driver for your source RDBMS installed. Any recent version of Windows comes with some ODBC drivers installed. For Windows 2000 and earlier these can be installed with the Microsoft Data Access Components (MDAC).

You should check if you have an ODBC driver for SQL Server. Start the Windows ODBC Data Source Administrator from MySQL Workbench using the **Plugins –> Start ODBC Administrator** menu item or just open a Windows terminal and type **odbcad32.exe**. Once there, go to the *Drivers* tab. You should see something like this:



As you can see, I already have two SQL Server ODBC drivers installed. The first one listed here (named *"SQL Server"*) comes preinstalled with Windows (you should have it as well). This driver is frozen at the level of functionality provided by SQL Server 2000 and it should be enough for you if your database doesn't make use of the new features and datatypes introduced after this SQL Server version. If you have a SQL Server instance in the same machine where you installed MySQL Workbench then you should also have the second driver listed in the image (named *"SQL Server Native Client..."*). This one comes with SQL Server and fully supports the companion SQL Server version. If you don't have it listed, you can download and install the Microsoft SQL Server 2012 Native Client. This is compatible with SQL Server 2012 as well as with previous SQL Server versions.

Once you take your pick on the driver to use, write down somewhere its name as shown in the ODBC Data Source Administrator. You'll need this name to connect to your SQL Server instance from the Migration Wizard. Let's go back to the Migration Wizard (you can close the ODBC Data Source Administrator now) and start the migration process.

Set up the parameters to connect to your source database

Click on the **Start Migration** button in the Overview page to advance to the Source Selection page. In this page you need to provide the information about the RDBMS you are migrating, the ODBC driver to use and the parameters for the connection.

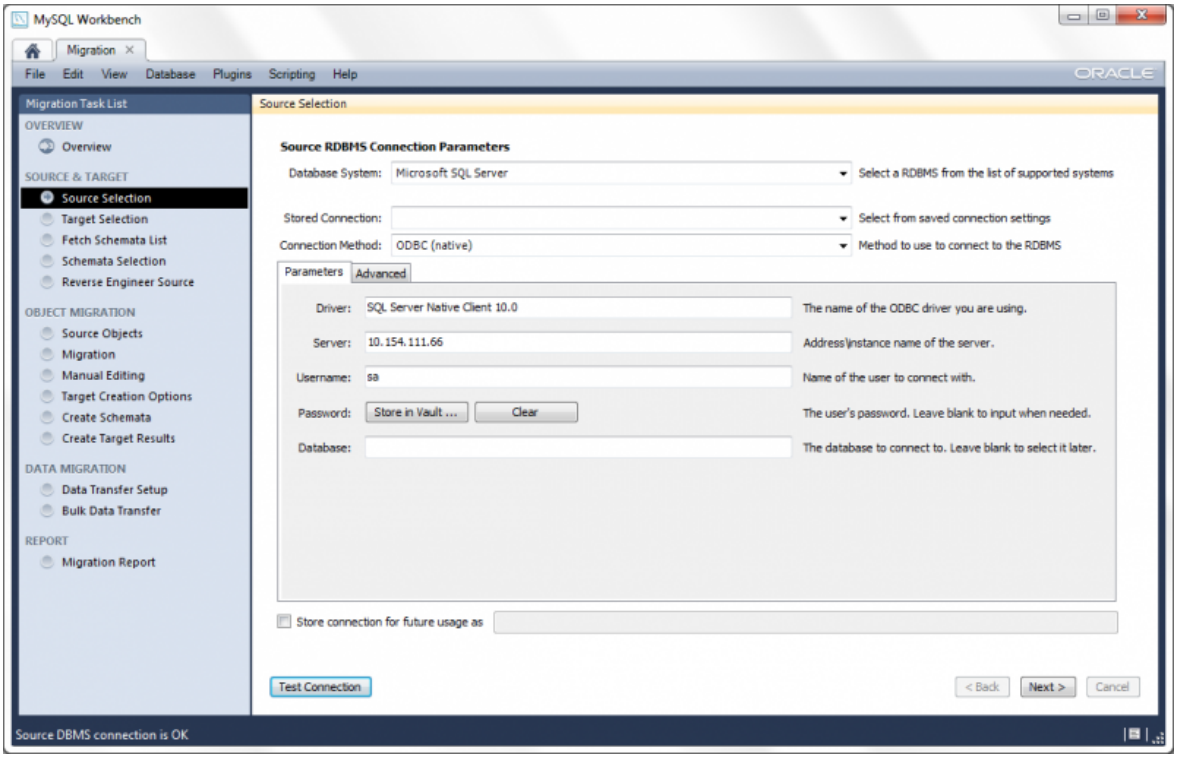
If you open the **Database System** combo box you'll find a list of the supported RDBMSes. Select *Microsoft SQL Server* from the list. Just below it there's another combo box named**Stored Connection.** It will list saved connection settings for that RDBMS. You can save connections by marking the checkbox at the bottom of the page and giving them a name of your preference.

The next combo box is for the selection of the **Connection Method**. This time we are going to select *ODBC (native)* from the list since we are using the native ODBC drivers provided by Microsoft. Other alternatives are ODBC data sources and ODBC FreeTDS (FreeTDS is a popular open source driver for Microsoft SQL Server and Sybase).

Now it's the time for putting the parameters for your connection. In the **Driver** text field, type the ODBC driver name from the previous step.

In the **Server** field put the values that identify your machine and your SQL Server instance name. If you don't recall these, go to SQL Server Management Studio and connect to your server. Then right click on the server icon in the Object Explorer and the name will be displayed in the new window that appears. If you have SQL Server Express Edition installed in your local machine and you haven't change the server name, then the default *"localhost\SQLEXPRESS"* should work for you. Another option is to put your server IP address instead of the host name. You can also specify a port by adding a comma after the server name/IP (E.g *"127.0.0.1,1433"*). The instance name is optional and defaults to the default SQL Server instance in the source host.

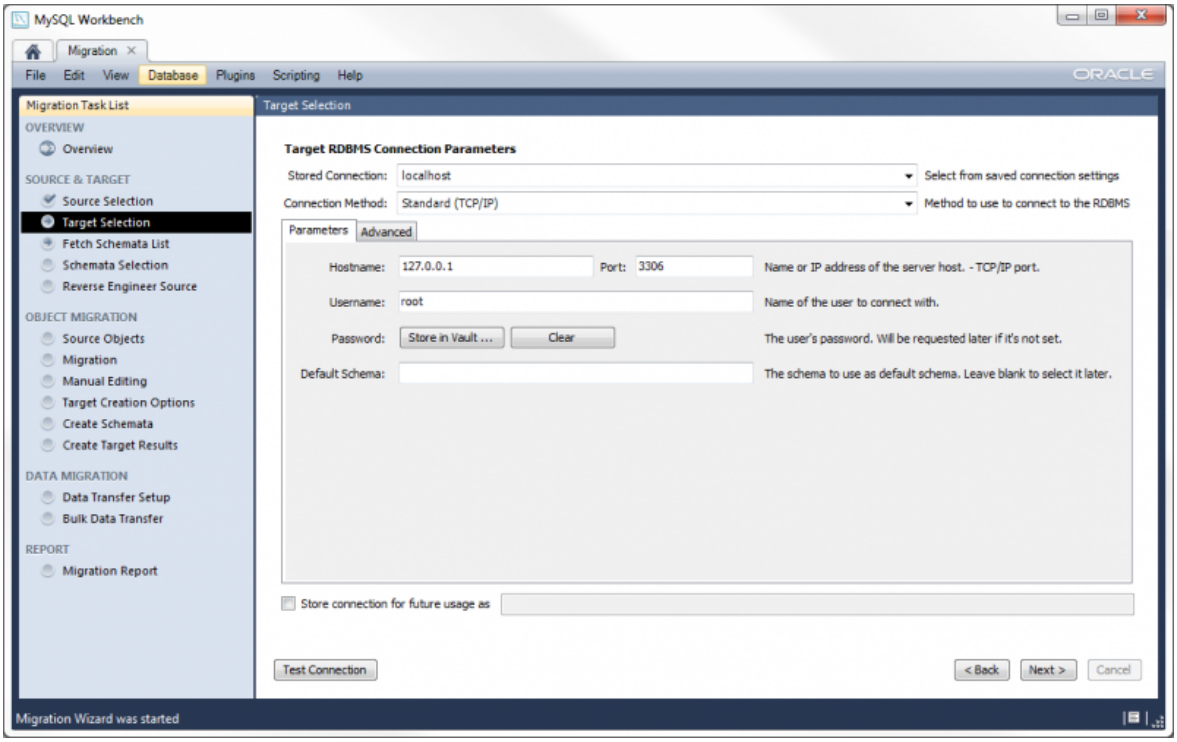
Now put your credentials (user name and password) to connect to the server. If you know the name of the database you want to migrate, put it in the **Database** field. Otherwise leave it blank and you will be later given a list to select your database there. At this point you should have something like this:



Click on the **Test Connection** button to check the connection to your SQL Server instance. If you put the right parameters you should see a message reporting a successful connection attempt.

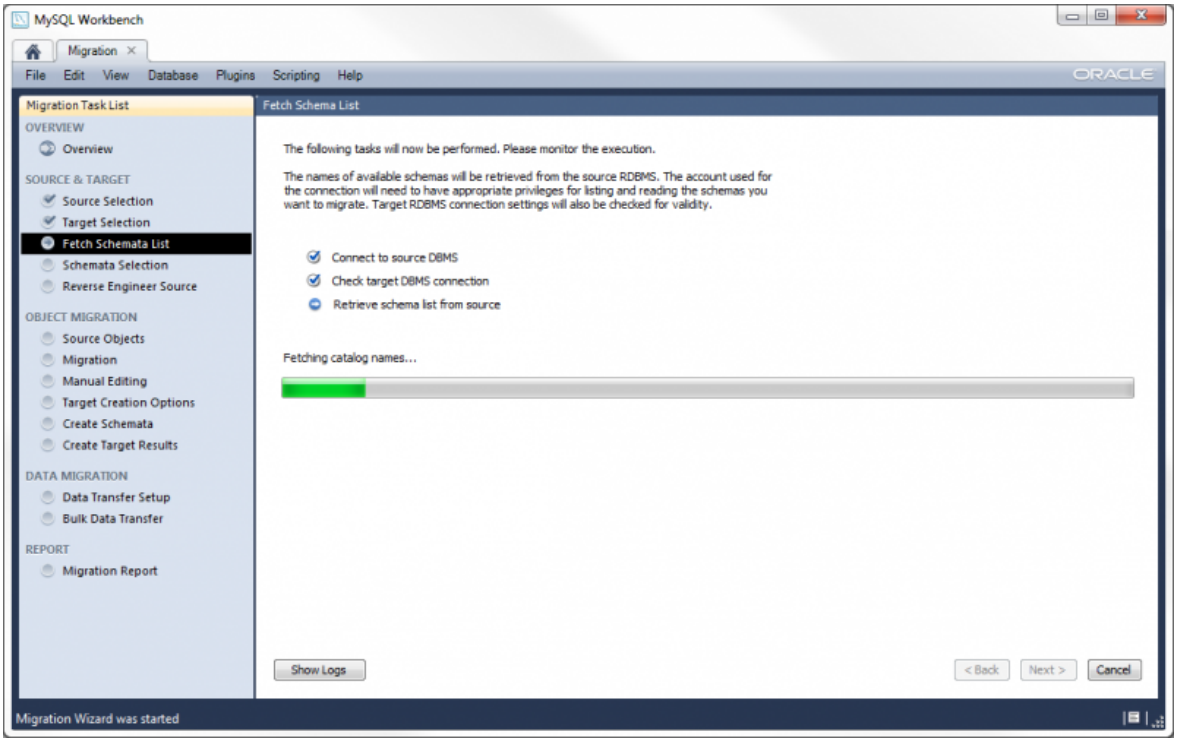
Set up the parameters to connect to your target database

Click on the **Next** button to move to the Target Selection page. Once there set the parameters to connect to your MySQL Server instance. When you are done click on the **Test Connection** button and verify that you can successfully connect to it.

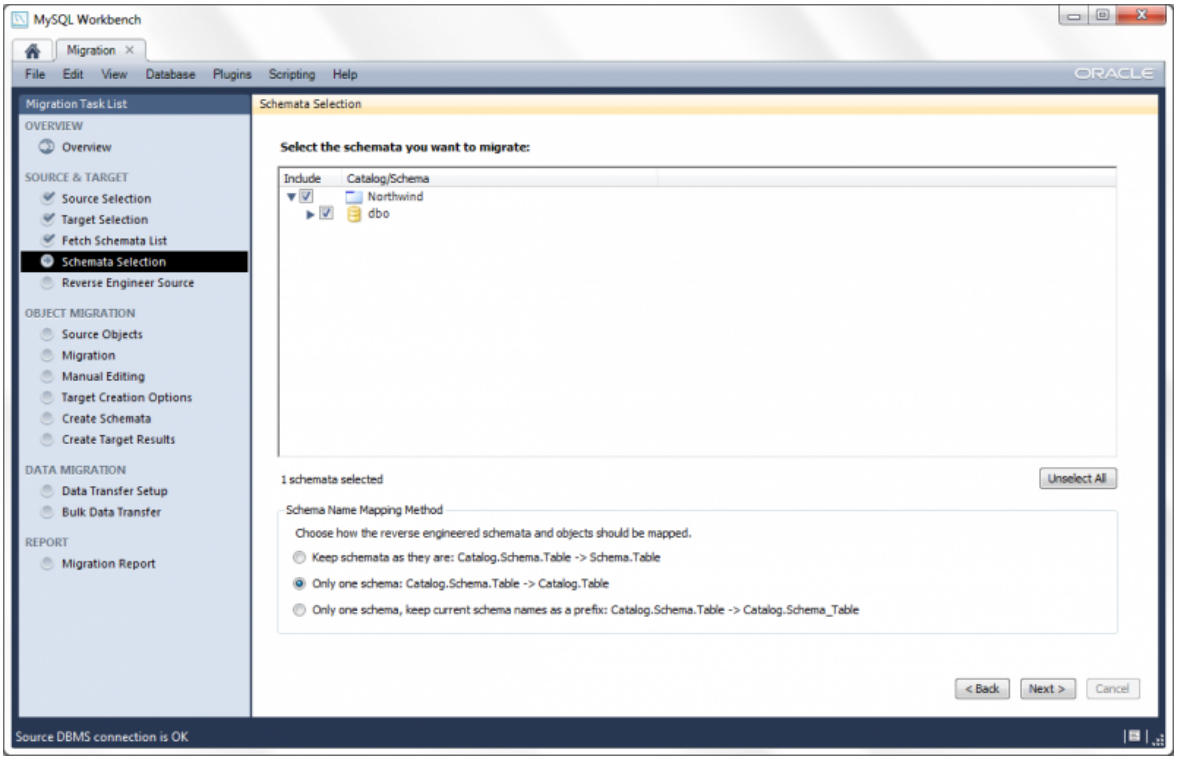


Select the schema(ta) to migrate

Click on the **Next** button to move to the next page. The Migration Wizard will communicate to your SQL Server instance to fetch a list of the catalogs and schemata. If you left blank the**Database** field in the Source Selection page it will retrieve all of the catalogs in the server. Otherwise it will just fetch the schemata corresponding to the catalog you explicitly typed.



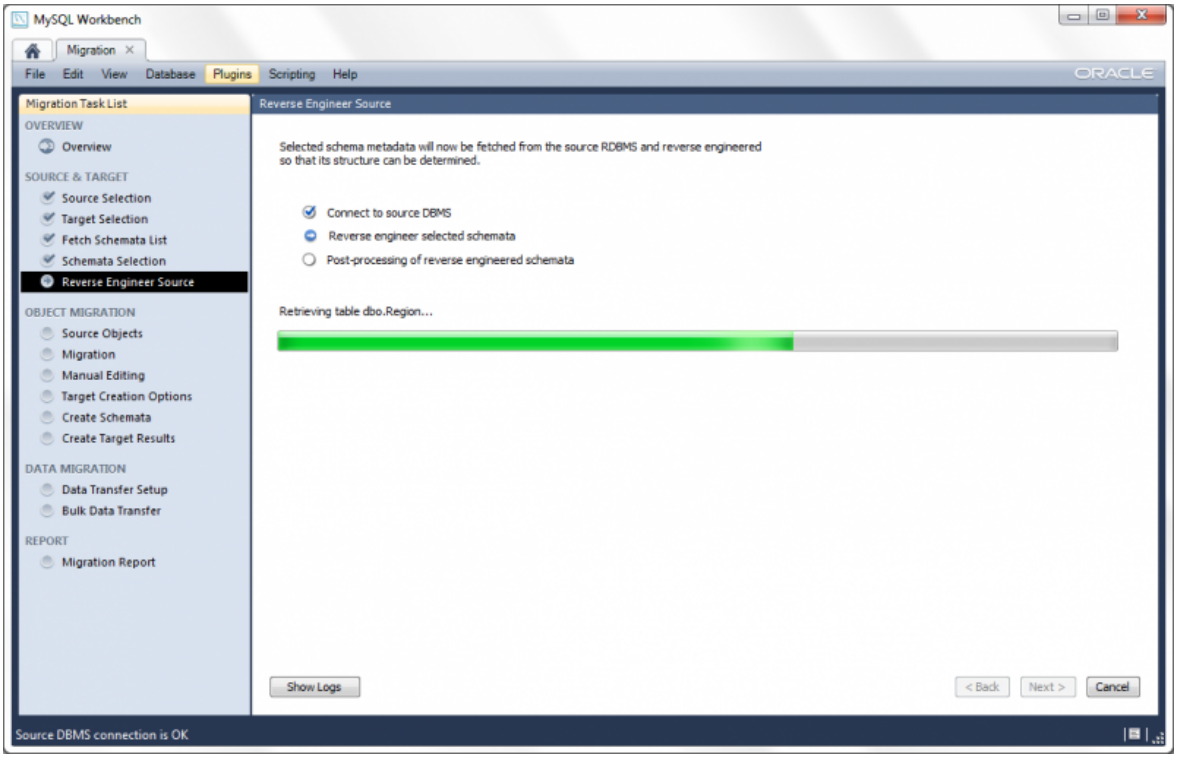
Verify that all tasks finished successfully and click on the **Next** button to move forward. You will be given a list of catalogs and their corresponding schemata to select the ones to migrate. Keep in mind that you can only migrate schemata from one catalog at a time. The Schema Selection page will look like this:



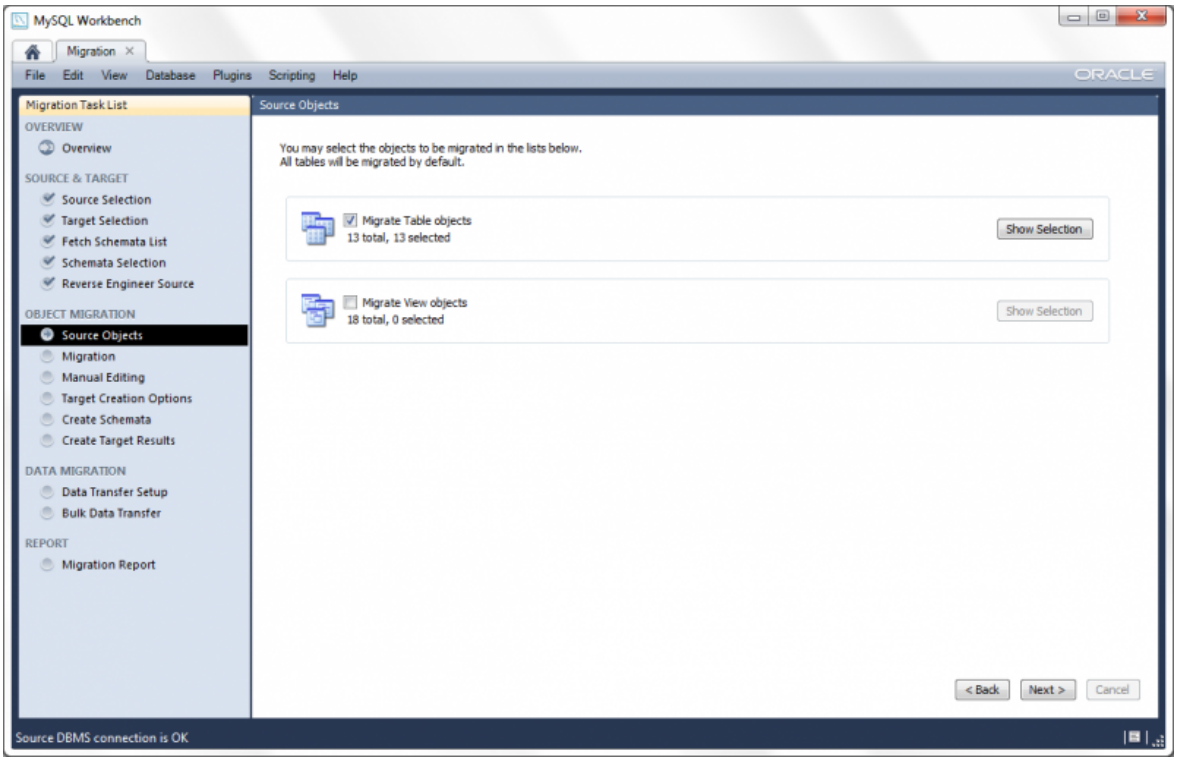
Select the Northwind sample database from the list and its default schema *dbo*. Now look at the options below. A SQL Server database is comprised of one catalog and one or more schemata. MySQL only supports one schema in each database (to be more precise, a MySQL database *is* a schema) so we have to tell the Migration Wizard how to handle the migration of schemata in our source database. We can either keep all of the schemata as they are (the Migration Wizard will create one database per schema), or merge them into a single MySQL database. The two last options are for specifying how the merge should be done: either remove the schema names (the Migration Wizard will handle the possible name colisions thay may appear along the way) or either adding the schema name to the database object names as a prefix. Let's select the second option since we only have one schema and we are not particularly interested in keeping its meaningless *dbo* name.

Select the objects to migrate

Move to the next page using the **Next** button. You should see the reverse engineering of the selected schema in progress. At this point the Migration Wizard is retrieving relevant information about the involved database objects (table names, table columns, primary and foreign keys, indices, triggers, views, etc.). You will be presented a page showing the progress as shown in the image below.

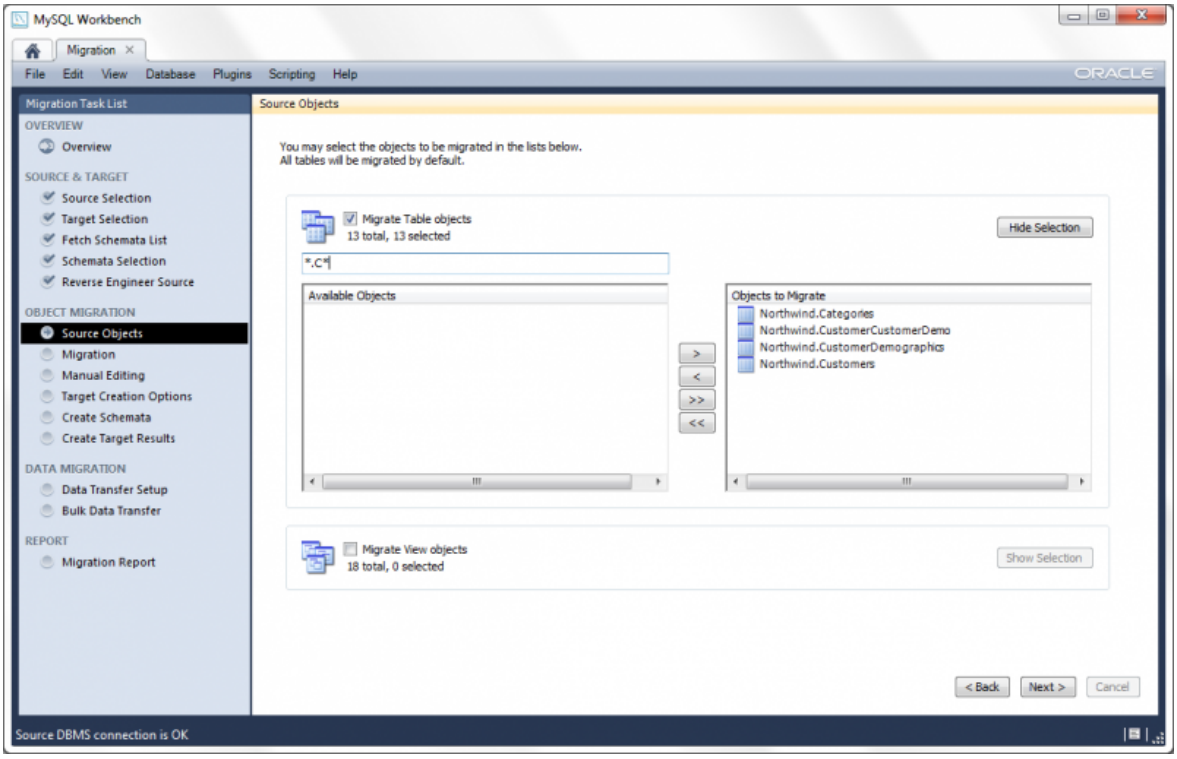


It may take some time, depending on how fast is your connection to the server, your SQL Server load and your local machine load. Wait for it to finish and verify that everything went well. Then move to the next page. In the Source Objects page you will have a list with the objects that were retrieved and are available for migration. It will look like this:



As you can see the Migration Wizard discovered table and view objects in our source database. Note that only the table objects are selected by default to be migrated. You can select the view objects too, but you would have to provide their corresponding MySQL equivalent code later (no automatic migration is available for them yet) so let's leave them off for now. The same applies for stored procedures, functions and triggers.

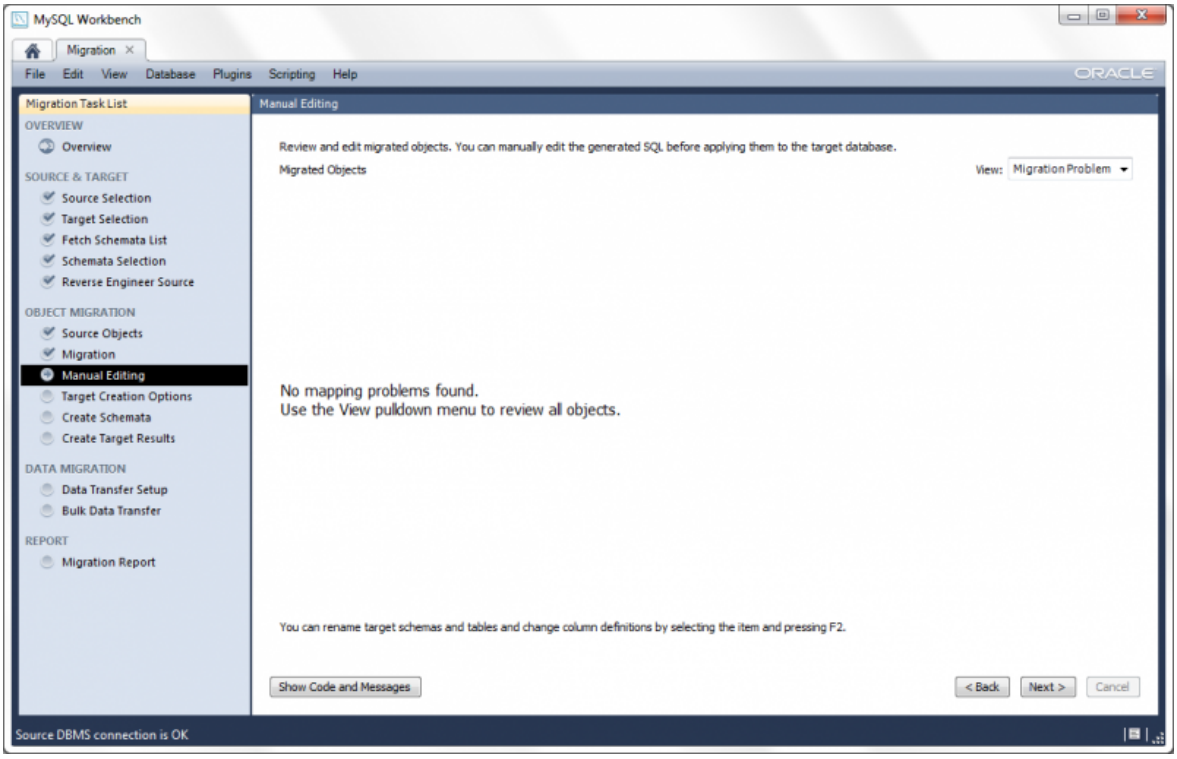
If you click on the **Show Selection** button you will be given the oportunity to select exactly which of them you want to migrate as shown here:



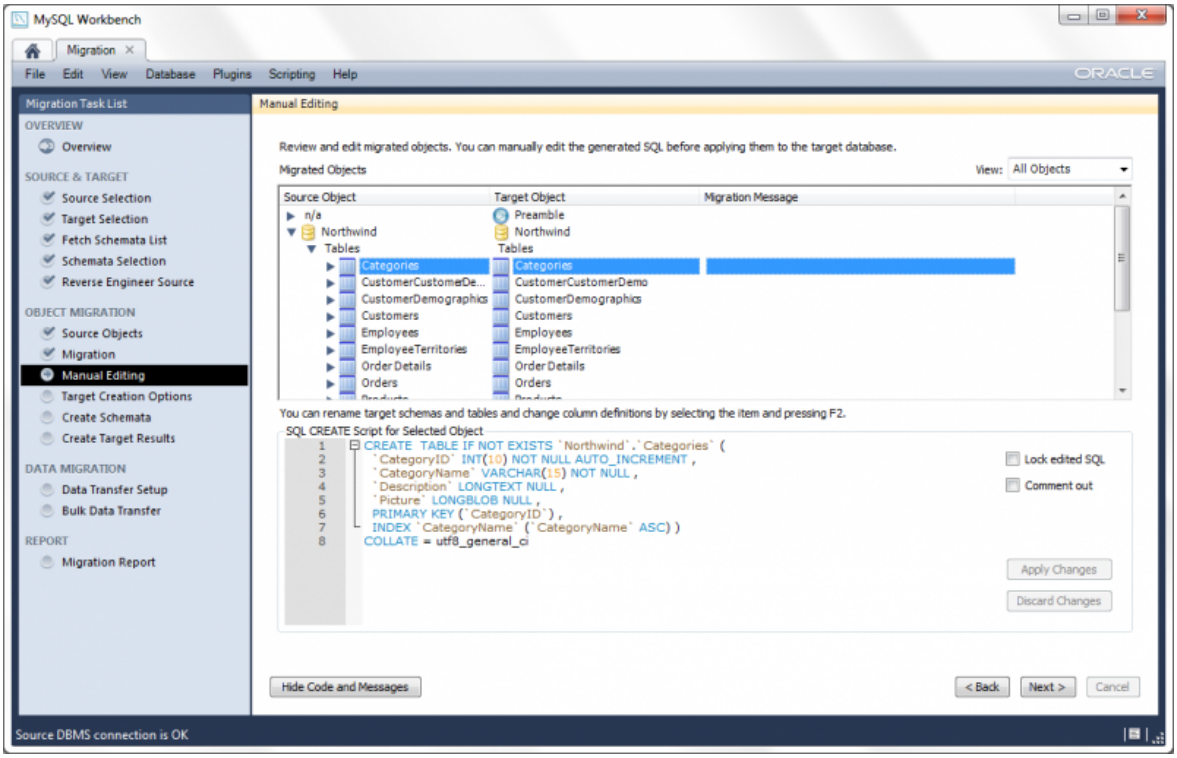
The items in the list to the right are the ones to be migrated. Note how you can use the filter box to easily filter the list (wildcards are allowed as you can see in the image above). By using the arrow buttons you can filter out the objects that you don't want to migrate. At the end, don't forget to clear the filter text box to check the full list of the selected objects. We are going to migrate all of the table objects, so make sure that all of them are in the **Objects to Migrate** list and that the **Migrate Table Objects** checkbox is checked. Most of the time you'll want to migrate all objects in the schema anyway, so you can just click Next.

Review the proposed migration

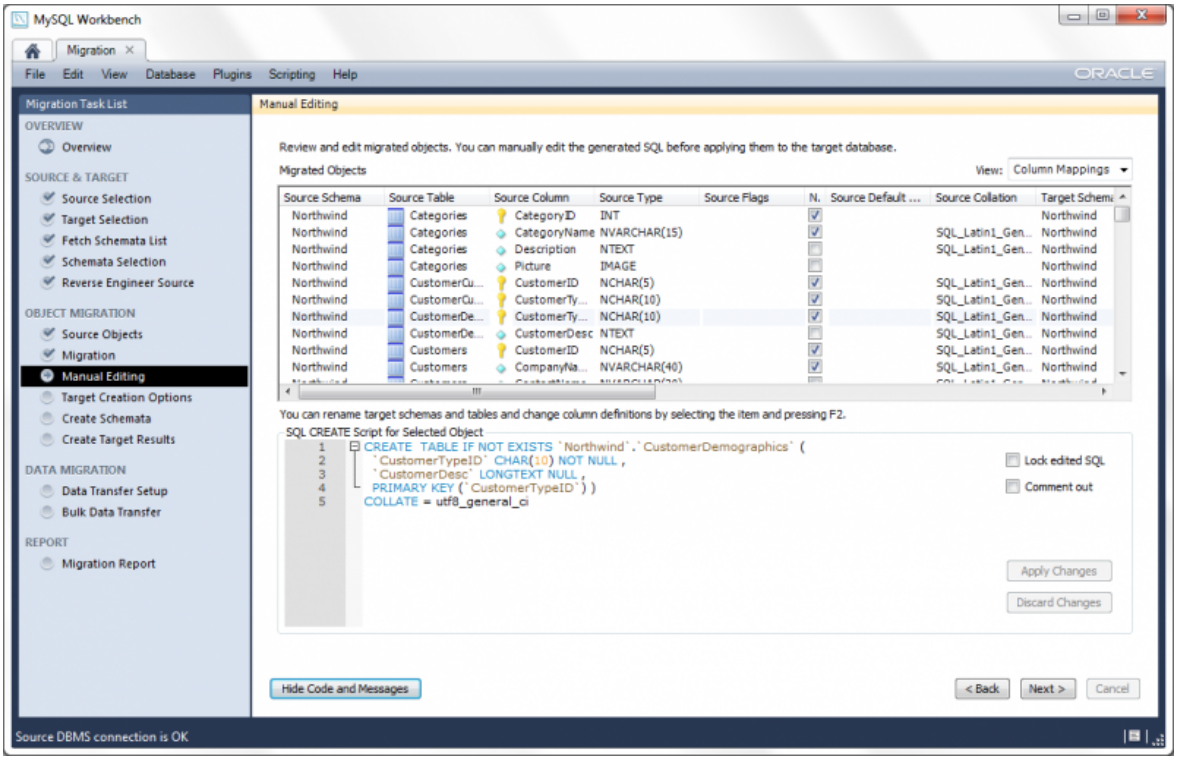
Move to the next page. You will see the progress of the migration there. At this point the Migration Wizard is converting the objects you selected into their equivalent objects in MySQL and creating the MySQL code needed to create them in the target server. Let it finish and move to the next page. You might have to wait a little bit before the **Manual Editing** page is ready but you'll end up with something like this:



As you can see in the image above there is a combo box named **View**. By using it you can change the way the migrated database objects are shown. Also take a look at the **Show Code and Messages** button. If you click on it you can see (and edit!) the generated MySQL code that corresponds to the selected object. Furthermore, you can double click in a row in the object tree and edit the name of the target object. Suppose you want your resultant database to have another name. No problem: double click on the Northwind row and rename it.

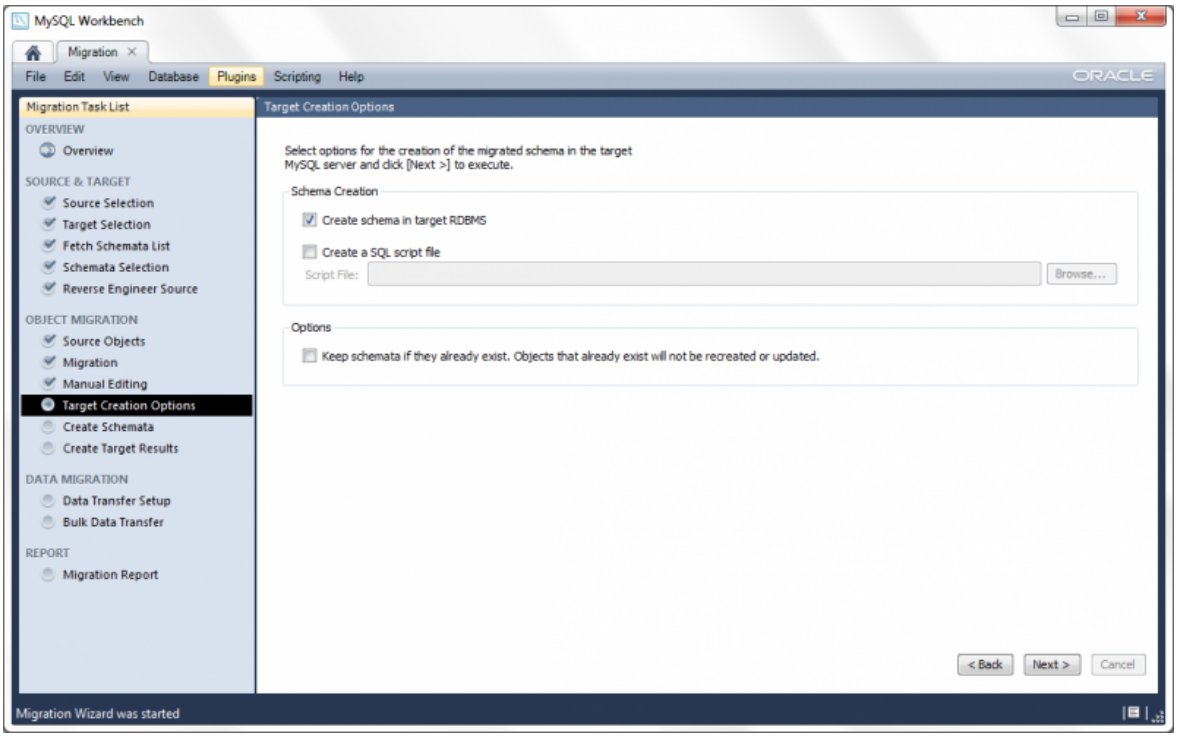


An interesting option in the **View** combo box is the *Column Mappings* one. It will show you all of the table columns and will let you individually review and fix the mapping of column types, default values and other attributes.

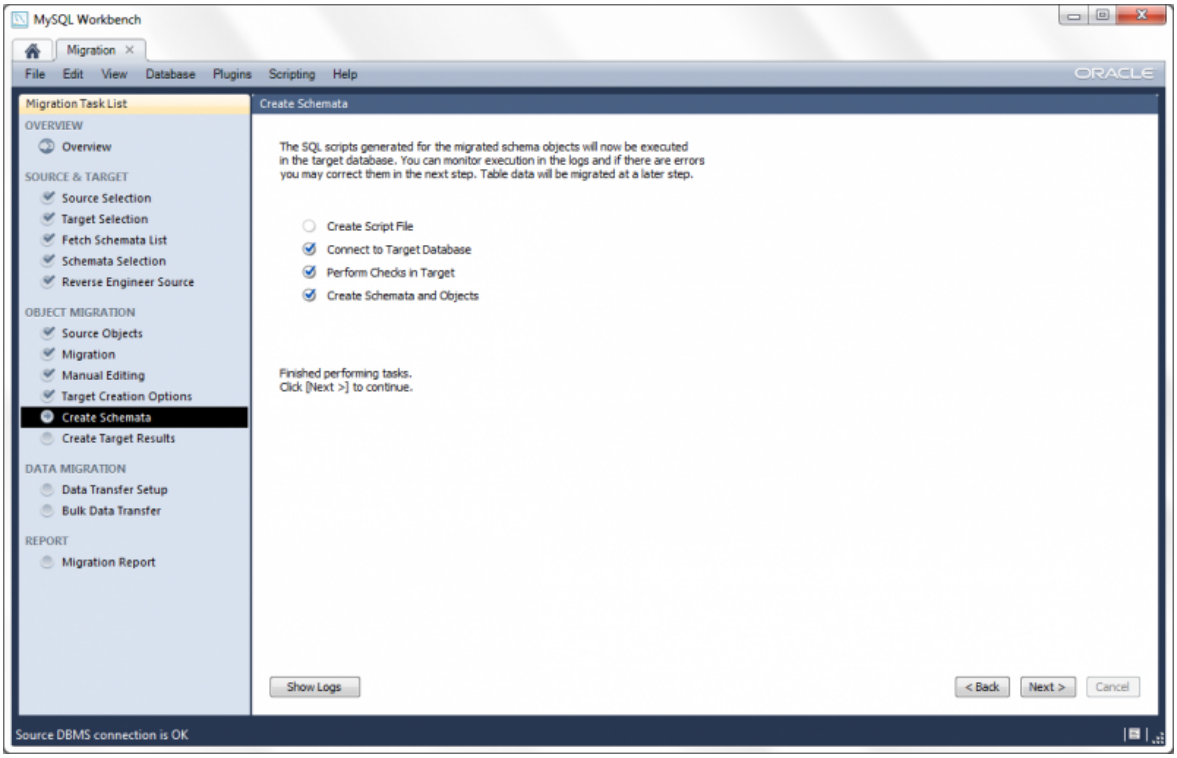


Run the resultant MySQL code to create the database objects

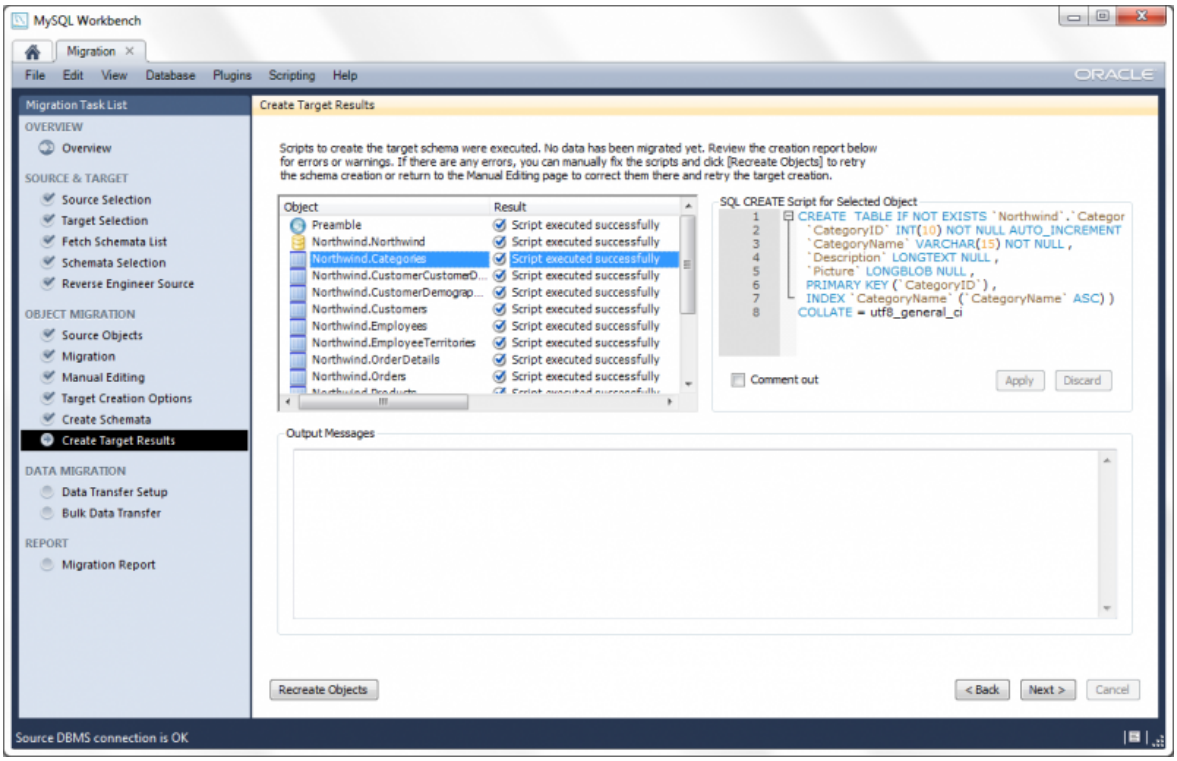
Move to the Target Creation Options page. It will look like this:



As you can see there, you are given the options of running the generated code in the target RDBMS (your MySQL instance from the second step) or just dumping it into a SQL script file. Leave it as shown in the image and move to the next page. The migrated SQL code will be executed in the target MySQL server. You can view its progress in the Create Schemata page:



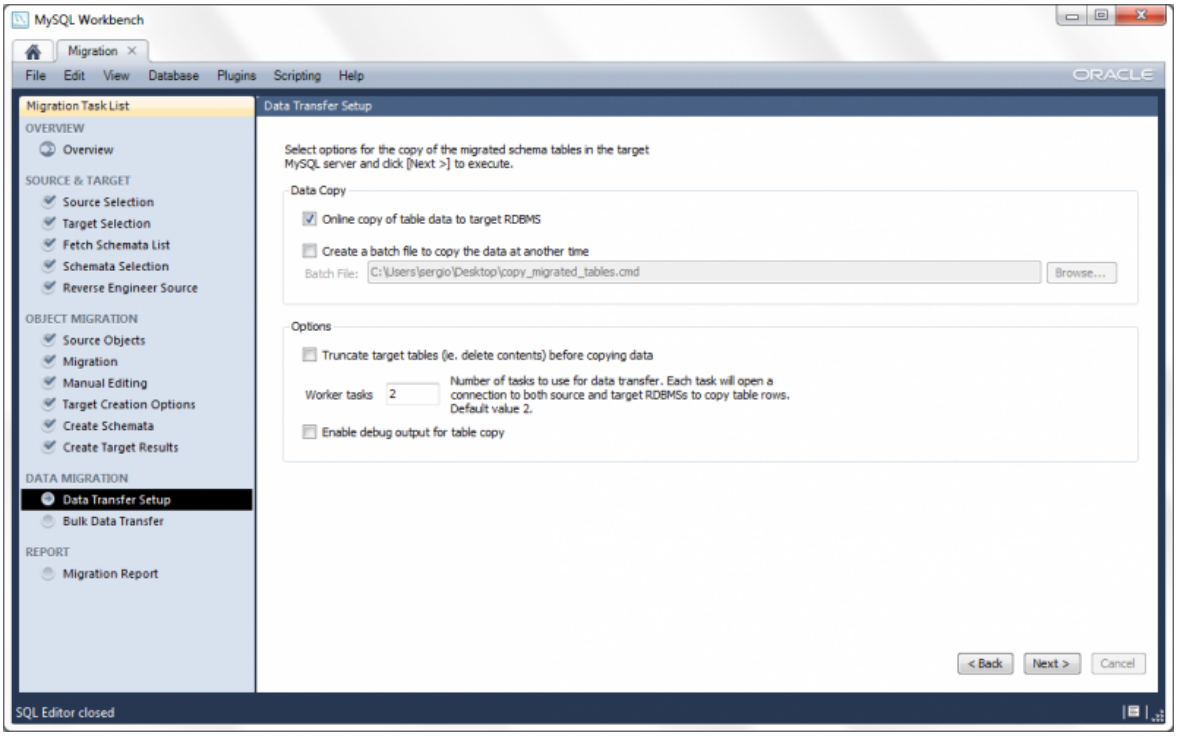
Once the creation of the schemata and their objects finishes you can move to the Create Target Results page. It will present you a list with the created objects and whether there were errors while creating them. Review it and make sure that everything went OK. It should look like this:



You can still edit the migration code using the code box to the right and save your changes by clicking on the **Apply** button. Keep in mind that you would still need to recreate the objects with the modified code in order to actually perform the changes. This is done by clicking on the **Recreate Objects** button. You may need to edit the generated code if its execution failed. You can then manually fix the SQL code and re-execute everything. In this tutorial we are not changing anything, so leave the code as it is and move to the Data Transfer Setup page by clicking on the **Next** button.

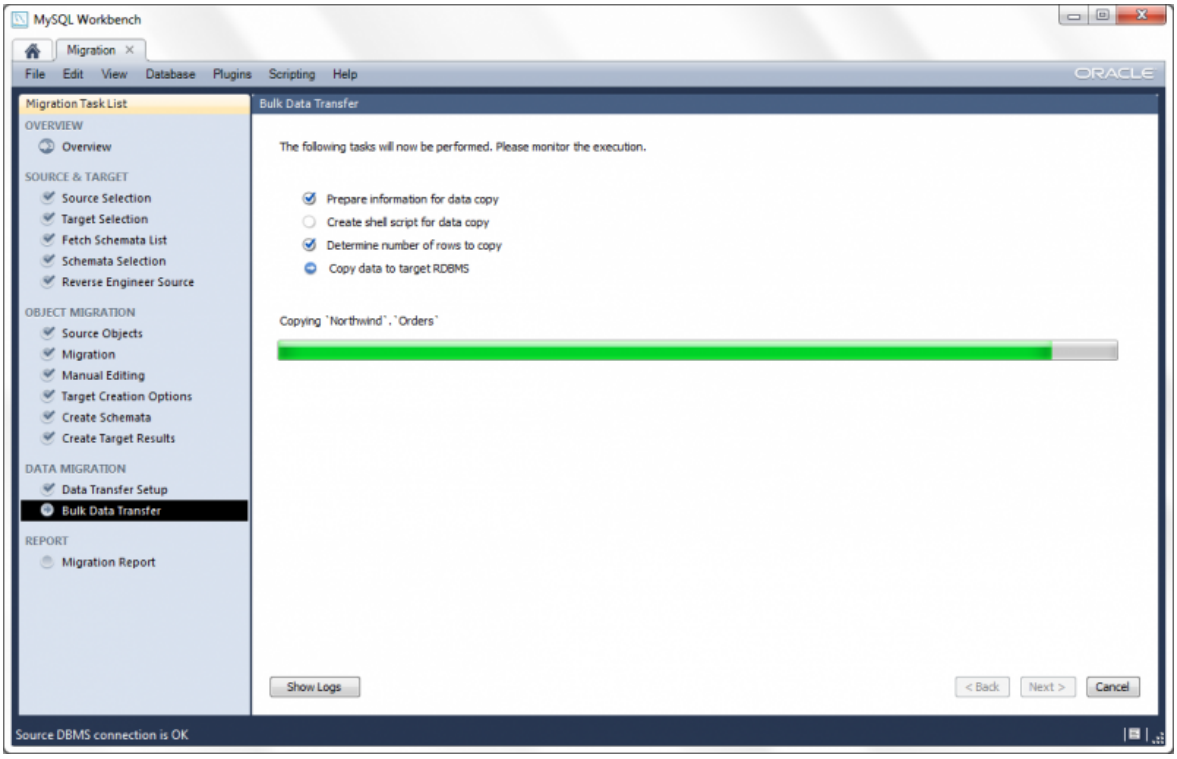
Transfer the data to the MySQL database

The next steps in the Migration Wizard are for the transference of data from the source SQL Server database into your newly created MySQL database. The Data Transfer Setup page allows you to configure this process.

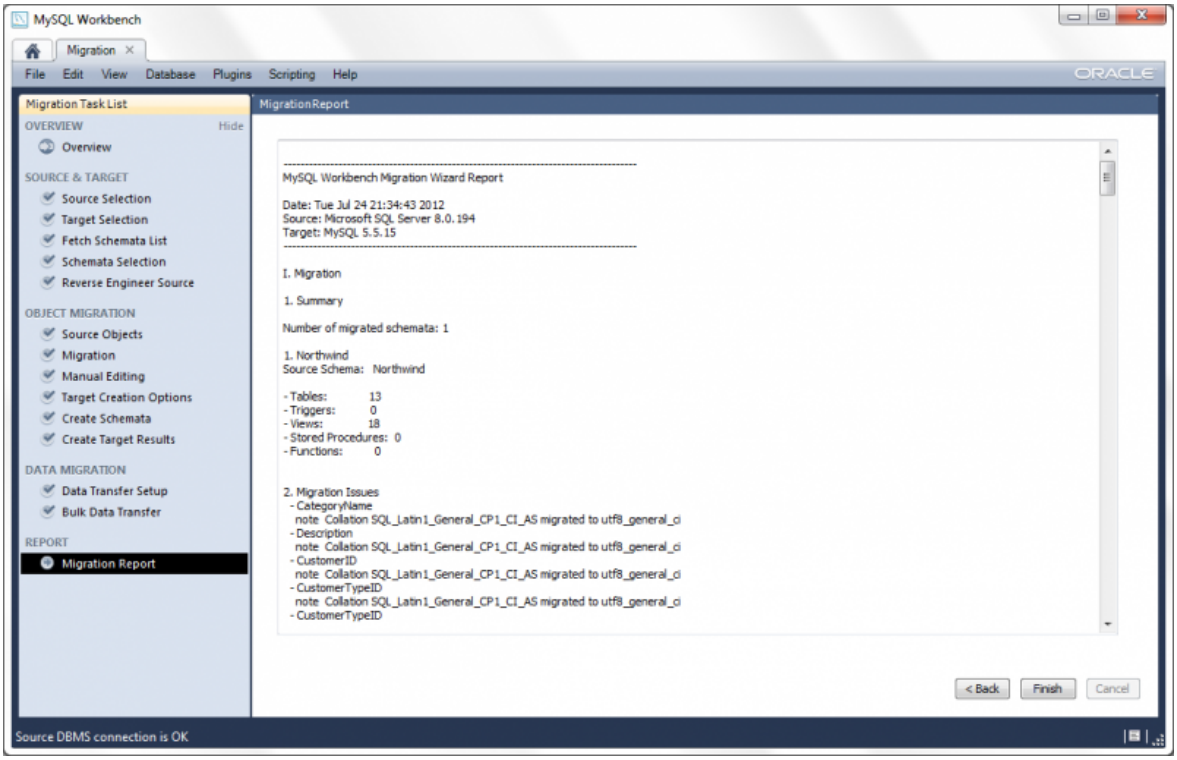


There are two sets of options here. The first one allows you to perform a live transference and/or to dump the data into a batch file that you can run later. The other set of options gives you a way to tune up this process.

Leave the default values for the options in this page as shown in the above image and move to the actual data transference by jumping to the next page. It will take a little while to copy the data. At this point the corresponding progress page will look familiar:



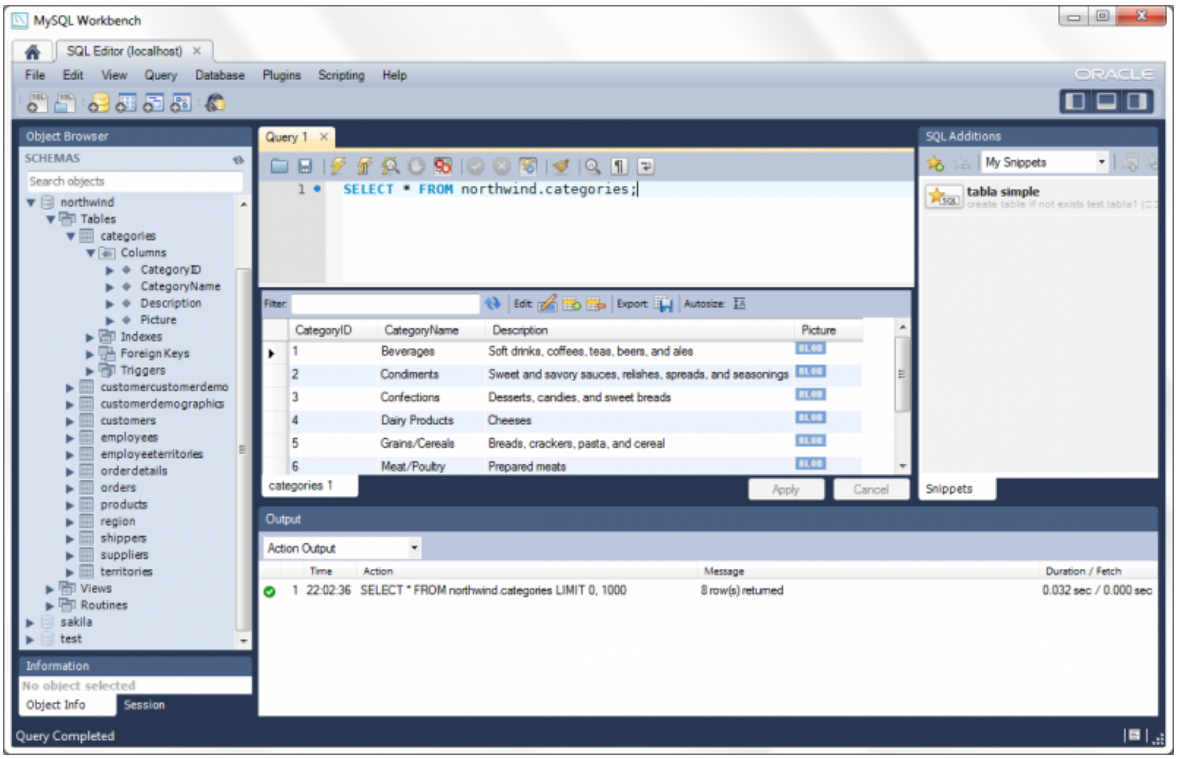
Once it finishes, move to the next page. You will be presented a report page summarizing the whole process:



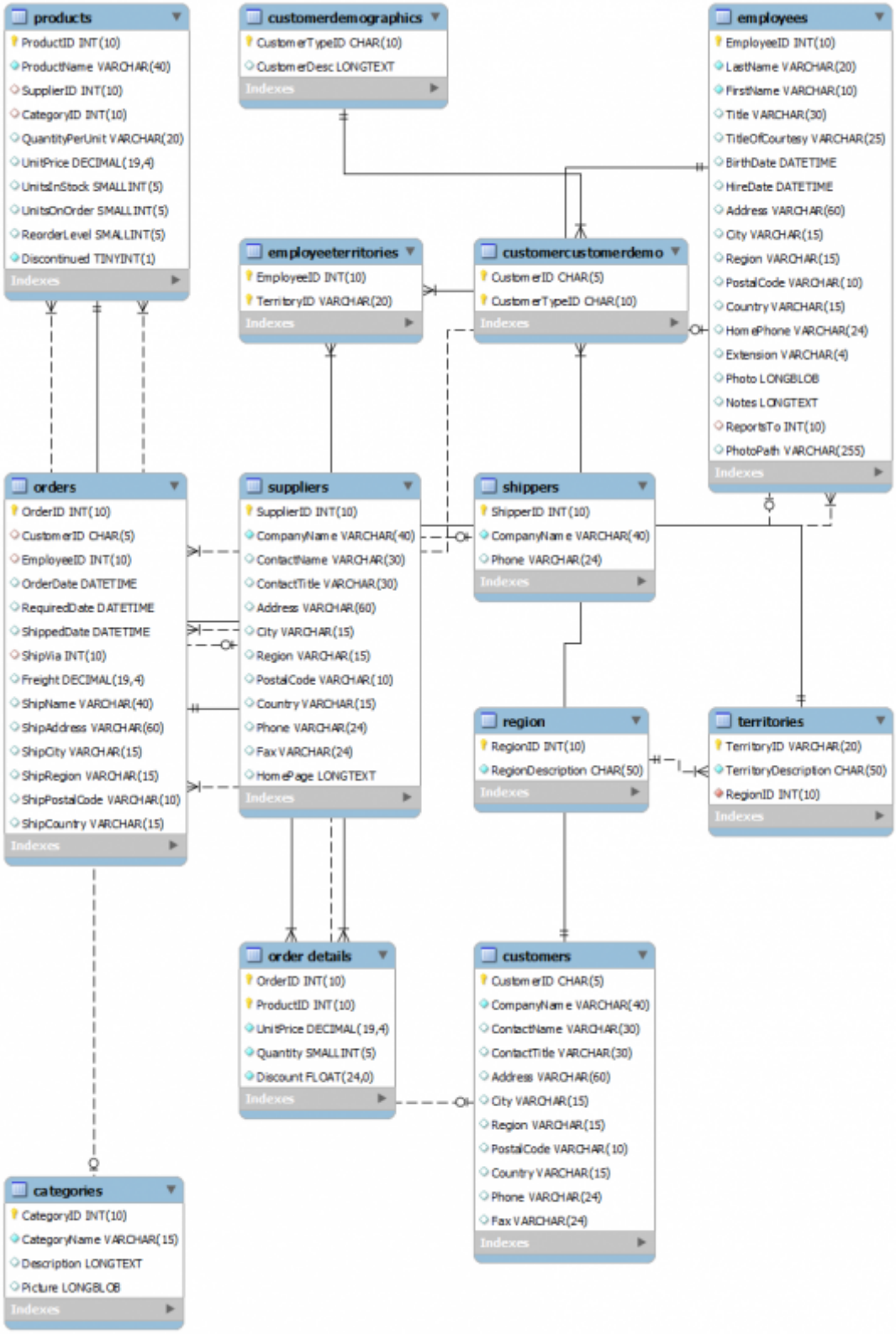
And that should be it. Click on the **Finish** button to close the Migration Wizard.

A little verification step

Now that the Northwind database was successfully migrated, let's see the results. Open an SQL Editor page associated with your MySQL Server instance and query the Northwind database. You can try something like *"SELECT * FROM Northwind.categories"*. You should get something like this:



And why not create an EER diagram from the migrated database? Click on **Create EER Model From Existing Database** in the main screen of MySQL Workbench and follow through the wizard steps. Once done go to **Arrange –> Autolayout** in the main menu to accomodate your EER model and you should get something like this:



Conclusions

By now you should have a pretty good idea of the capabilities of the Migration Wizard and should be ready to use it for your own migrations. The official documentation is also

there for you and you can always ask any question in the comments of this post or in the migration official forum. Live long and prosper!

Sergio de la Cruz

Milosz Bodzek / July 25, 2012 / Migration, Tutorial, Windows

---

17 thoughts on "How-To: Guide to Database Migration from Microsoft SQL Server using MySQL Workbench"

Pingback: MySQL Workbench 6.2 » Blog Archive » Installing a driver for Microsoft SQL Server and Sybase ASE in Linux and Mac

mensolution.dk

June 4, 2015 at 7:55 pm

Hello to every one, the contents present at this site are really awesome for people knowledge, well,
keep up the nice work fellows.

??????????????????????

June 4, 2015 at 10:42 pm

I drop a comment when I appreciate a post on a site or if I have something to contribute to the conversation. Usually it is triggered by
the sincerness displayed in the article I read. And on this article MySQL Workbench 6.3 ? Blog Archive ? How-To: Guide to Database Migration from Microsoft SQL Server using MySQL Workbench.
I was actually excited enough to drop a thought 😛 I do

have some questions for you if it's okay. Is it only me
or do a few of these comments look like they are
left by brain dead visitors? 😛 And, if you are writing at other sites, I would like to keep up with you.
Could you list every one of all your social sites like your twitter feed, Facebook page or linkedin profile?

Azizi

September 1, 2015 at 11:03 am

Hi, I'm just wondering, can we migrate only one table from mssql to mysql? Because I only need one table, not the whole database.

Milosz Bodzek

September 1, 2015 at 2:46 pm

Yes, of course you can migrate only one table. In 'Source Object' step click on 'Show Selection' button to expand selected tables. Now you can easily select only table you want by using buttons.

---

oralce

September 19, 2015 at 3:35 am

From Oracle ,win8 64bit , step4 scheme list still is null

Umang

November 19, 2015 at 12:56 pm

Can we Migrate only selected stored Pocedure from one server to another and not migrating the whole database.

Thanks

Mike Lischke

November 19, 2015 at 1:13 pm

Umang, please don't ask support question on the blog. Rather use our forum or Q&A sites like http://www.stackoverflow.com.

Jesus Angeles

November 23, 2015 at 11:12 am

You are the best!

---

aathavan

May 3, 2016 at 4:30 am

how to replicate sql server to mysql databse ?

Mike Lischke

May 3, 2016 at 7:55 am

Same for you aathavan: a blog is not a support medium.

---

christina

July 3, 2016 at 6:04 am

how can i migrate from a local ms sql it will not authenticate please help

Milosz Bodzek

July 4, 2016 at 6:59 am

christina: please use our forum (http://forums.mysql.com) to get support.

Earl Wertheimer

July 9, 2016 at 8:44 am

Useless. After 6 hours of dicking around, I converted about 40 of 150 tables.
No errors, nothing.
Moving on to Toad…

Milosz Bodzek

July 11, 2016 at 9:25 am

Better use our forum at http://forums.mysql.com/list.php?104 to ask questions and get help.

Earl Wertheimer

July 11, 2016 at 11:26 pm

Here is the problem: SQLServer tables with many columns will cause an error in bcp due to the query length (~8000 chars) limit.
That's because it adds a CAST([field] as NVARCHAR(len)) as [field] to every text field.
Even if the source collation is Latin1 going to a UTF-8 MySQL database.
Once 'bulk_copy_tables.cmd' crashes, all subsequent tables are ignored.

The procedure should check the bcp query length or make the CAST(optional.

Milosz Bodzek

July 12, 2016 at 6:46 am

Thank you for your comment, but use rather our forum at http://forums.mysql.com/list.php?104 for technical issues.